

# Actividad N° 4 – Tema 1

## " Pilares de Calidad en Desarrollo de Software "

**Objetivo:** El objetivo de esta actividad es que los alumnos internalicen y fundamenten la importancia de los principios del Código Limpio (Clean Code), entendiendo que la calidad del código impacta directamente en la mantenibilidad y la velocidad de desarrollo a largo plazo.

---

### Instrucciones Generales

Esta actividad está diseñada para que los equipos realicen una investigación profunda sobre los fundamentos teóricos del **Código Limpio**.

- **Formato de Entrega:** Un **documento PDF** que contenga las respuestas claras y concisas a las preguntas guía, citando las fuentes consultadas.
  - **Colaboración:** El trabajo en grupo es fundamental. Debatan y tomen decisiones conjuntas sobre la interpretación y las respuestas para asegurar un análisis robusto.
  - **Plazo de Entrega:** La fecha límite para la entrega de la actividad es el **5 de Octubre**.
- 

### Descripción de la Tarea (Investigación y Fundamentación)

Los alumnos deberán realizar una investigación sobre los principios del Código Limpio y responder las siguientes preguntas guía con un enfoque conceptual y de alto nivel.

#### *1. El Principio de la Claridad y la Mantenibilidad*

- **Definición Esencial:** Investigue y defina, en términos sencillos, qué significa que el código sea "**limpio**". Explique por qué el objetivo principal del Código Limpio no es el rendimiento de la máquina, sino la **facilidad de lectura y comprensión** humana.
- **El Costo Oculto:** Explique cómo el código sucio o desorganizado se convierte en "**código espagueti**" (*spaghetti code*) y cómo este dificulta que otro desarrollador (o usted mismo en el futuro) pueda **modificar la lógica rápidamente** sin causar un error.

## 2. Nombres Significativos (Nombres que Hablan)

- **Propósito Revelador:** Investigue y ejemplifique el principio de **Nombres Significativos** (variables, funciones, clases). ¿Qué significa que un nombre debe "revelar el propósito"?
- **Comentarios de Reemplazo:** Explique por qué los comentarios que solo sirven para explicar qué hace una variable (ej. `// c es el contador`) son considerados una **mala práctica** en el Código Limpio. Muestre un ejemplo de un **nombre malo** (ej. `c`) y proponga un **nombre claro** que elimine la necesidad del comentario (ej. `contadorClientes`).

## 3. Funciones Pequeñas y con Responsabilidad Única

- **Responsabilidad Única:** Investigue el principio de que las funciones o métodos deben ser "pequeños y hacer una sola cosa" (*Single Responsibility Principle*).
- **Anatomía de la Función Larga:** Use la analogía de un **proceso complejo** (ej. una receta, un trámite bancario, etc.) para explicar por qué una función larga que mezcla cálculos, conexiones a base de datos y formato de datos es difícil de seguir, modificar y **probar**, a diferencia de dividir el trabajo en funciones más pequeñas y especializadas.